

Studio Manette "ManetteTools" - [Partner Doc]

Mots clés :

- Documentation ManetteTools for Blender
 - Presentation
 - General principle
 - Assets production process
 - Shots production process
 - Layout
 - Animation
 - Render
- Install
 - Tools Installation
 - Step by step:
 - Banks installation
 - Shaders
 - Animations
- File management
 - Server environment
 - l'arborescence - tree structure
 - pour les assets:
 - pour les shots:
 - Shots:
 - Backup - Backup
 - Versionning:
 - Versioning:
 - Fichiers de sauvegarde et d'exports:
 - Backup and export files:
 -
 - configuration blender - blender config
- Nomenclature et fichiers - Nomenclature and files:
 - la nomenclature - nomenclature
- process communs (common processes):
 - tasks:
 - exports manuels:
 - publishing:
 - première utilisation / changement de production:
- tools communs:
- process de fabrication des assets:
 - pré modeling:
 - review sup:
 - vie de l'asset:
- tools de modeling:
 - Tools Panel — modeling tools
 - add simple master for all bg assets
 - add master for selected assets
 - merge character model
 - convert to prop
 - clone selected assets

- switch to rigged asset
- force confo
- remove assets
- replace bg data copies
- Tools Panel — BG tools
 - push blendshape on original
 - open original asset
 - build BG from BGelements
- Tools Panel — proxy tools
 - replace geo by primitive
 - replace geo by selection
 - replace geo by decimate
 - replace all geo by decimate
 - convert asset to proxy
- Tools Panel — combo tools
 - create combo asset
 - update combo asset
 - open combo asset
- process de layout:
- tools de layout:
- process d'anim:
- tools d'anim:
- task panel:
 - tasks
 - open
 - share
 - save
 - render
 - send
- pour les productions entièrement sur aquarium:
 - look for media and notes:
 - send timelog
 - change day
- modeling panel:
 - add master for all bg assets:
 - add master for selected assets:
 - clone selected props:
 - replace data copies:
 - export selected props:
 - player:
- rigging panel:
 - clean rig:
 - drivers by default
 - reglages des articulations.
 - symmetry drivers L -> R
 - symmetry drivers R -> L
 - curves by default
 - drivers on selected bone
 - save drivers param
 - load drivers param
 - export character

- layout panel:
 - update
 - populate
 - populate all
 - open database:
 - create rigged camera:
 - export setdressing:
 - import setdressing:
 - mbx export material:
- anim panel:
 - interface anim:
 - update characters list:
 - cbx export anim:
 - cbx import anim:
 - export facial anim:
- versions panel:
 - change version:
- nomenclature rig:
- annexes:
 - schéma de création des background:
 - background creation scheme:
 - vocabulaire:
 - fonctionnement des collections:
 - notice de creation des json des rigs
 - descriptif des scripts composants le pipe:
 - le fonctionnement des mises à jour auto de la base:



This is a **Work in Progress** documentation.

Documentation ManetteTools for Blender [↗](#)

Presentation [↗](#)

General principle [↗](#)

- The Manette pipe is a set of tools for creating cartoons using Blender and Unity software. It only integrates the necessary tools in Blender, the Unity part being entirely managed by Manette's internal team.
- It works to a certain extent like compositing or editing software: as in a compositing file where the media is not included, the shot save does not contain the assets, but just the list of uses made of them.
- The elements we generate are of 2 types:
 - **Assets** (props, bg, characters, etc...): they correspond to the media. Each exists in 2 forms: blender files and export files (.fbx and .mbx).

- **Shots** (layout, anim): they are the description of the assets and their transformations in time and space for a given shot. They are built from a file describing the scene (default position of all assets, shot duration)(.sbx), and as many animation files (.cbx) as there are rigs in the scene, describing their respective animations. Finally, a materials file (.mbx) describing the material modifications of all assets in the scene (optional).
- The operation is the same at each stage of the shot process (layout, anim, fx, rendering) and under the 2 softwares (blender and unity):
 - Reconstruction from the data (fbx,mbx,sbx,cbx)
 - Generation and export of new data in addition to the data received.
- The only difference between the blender (layout,anim) and unity (fx,rendering) process is in the use, only in blender, of .blend files for rigged assets, in order to be able to animate them.

The animation save format (.cbx) is common to blender and unity. You can save either the animation of the controllers (which can be replayed and modified in blender), or that of the deformers (which can be replayed in unity and blender but not modified). A simple rig, where the controllers are also deformers, can be read, modified and written from Blender like Unity.

- These data from a shot is not encapsulated in a single file but has its own files, allowing easy updates, replacement, and mixing of data from other shots.
- Finally, these data (with the exception of the .fbx and .blend formats) are in ascii, greatly facilitating their opening, understanding and corrections if necessary with a simple text editor, without requiring blender or unity.

Assets production process [↗](#)

- step 1: props and bg-éléments production
- step 2: objects in a background production
- step 3: import of props and bg-elements needed to complete the bg
- step 4: set up, deformation, adaptation
- step 5: set up root and connexions to another background
- step 6: optimization
- step 7: data export

Shots production process [↗](#)

Layout [↗](#)

prep: for each sequence, we must determine a shot that will serve as a reference for the organization of the different bg required. This worldmap is used for each shot in the sequence.

- step 1: construction of the layout from the data
- step 2: adaptation of the assets
- step 3: camera and character animation (for reference and for the framing)
- step 4: export of the set-dressing and animation of the shot.

Animation [↗](#)

- step 1: reconstruction of the layout from the data

step 2: animation of the characters and props.

step 3: export of the set-dressing and animations.

Render

step 1: reconstruction of the animation from the data

step 2: addition fx and lighting.

step 3: rendering

Install

The complete installation need the following environnement:

- a server, synchronized on Caribara's production server. It must be accessible from all workstations in order to be able to exchange data with production and other studios.
- a server specific to the studio, accessible from all the workstations within it, to be able to exchange data between operators and supervisors..
- an additional machine (without operator) to be able to run automated processes, also accessing the servers.

On all these machines, the following software must be installed:

- XnView
- Notepad++
- Tortoise SVN
- Ffmpeg
- VLC
- Blender 3.1.2

On each, the backup space used by the ManetteTools is a virtual hard disk that must be created and named X: , the caribara server S: and the studio server F: (on SMJ). ([voir schéma](#))

Tools Installation

The tools are provided with Tortoise SVN. It's here : Z:\ManetteTools\ManetteTools_setup.exe

(Z is in this context a virtual hard disk mounted from the directory where the checkout was done)

- Manette Tools is like a executable to launch. They do not require administrator rights to be installed. They will be installed at:

C:\Users*username*\AppData\Roaming\Blender Foundation\Blender\3.1\scripts\addons

- They require the prior installation of a version of blender 3.1.2 (default install)

Apart from controller tools, the only addon used is rigify (rigging tool). It's in Blender by default.

Step by step:

1. please, install blender, ffmpeg , xnview, vlc and notepad++

1. The Manette_work folder will be the folder where your works will be saved. please copy it where you want on your pc.

3)in this folder there's a mountX.bat file. execute it in administrator. now, tools can access it directly with the path X:

you can put a shortcut of this .bat file in your starting menu to automatically mount the X drive when you start your computer.

1. install the ManetteTools

1. open blender, configure the preferences and activate the ManetteTools ([see details](#)) , close blender.

1. re-open blender, in the manette tools, log in, select the production and click on "set default log".

Banks installation

Shaders

You must have referenced the shader library by adding this path in the "Asset libraries":

S:\99_TAKES\03_ASSETS\07_TEXTURES\07_SHARE\Library

Animations

there's a ".bat" file in each of these folders. To mirror these banks locally, execute each one.

- S:\99_TAKES\05_bank\ANIMATION_BANK
- S:\99_TAKES\05_bank\POSE_LIBRARY

you must have referenced the animation libraries by adding theses paths in the "Asset libraries":

- X:\01_POSE_LIBRARY
- X:\02_ANIMATION_BANK

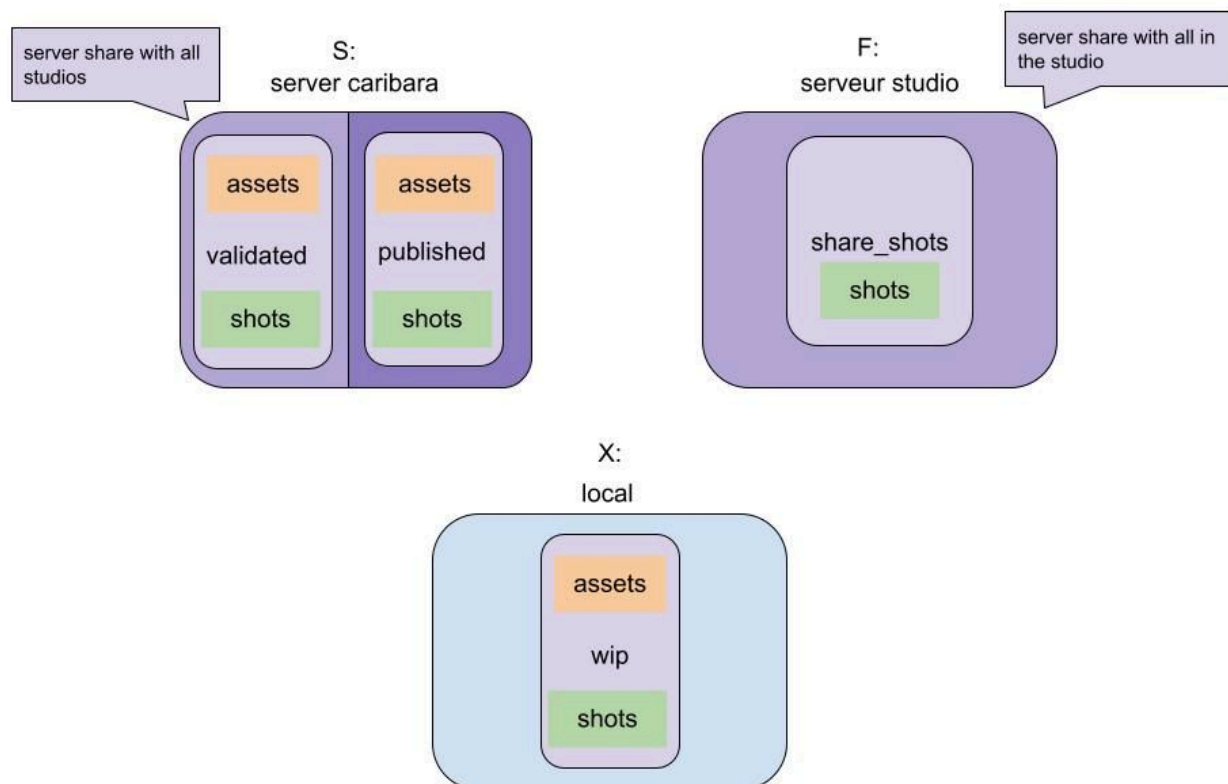
You can put a copy of the two .bat files in your starting menu to update automatically the banks when you restart your computer.

File management [↗](#)

Server environment [↗](#)

The data is divided into several spaces:

(the names and letters are those used on SMJ)



- X: local virtual hard disk on each machine
- F: directory shared with the whole studio
- S: directory shared with all production
- X contient toutes les sauvegardes de chaque utilisateur des fichiers sur lesquels il a travaillé.

F contains a SHARE_SHOTS directory which is used to share and centralize the files before submission to the director or to present to the supervisor.

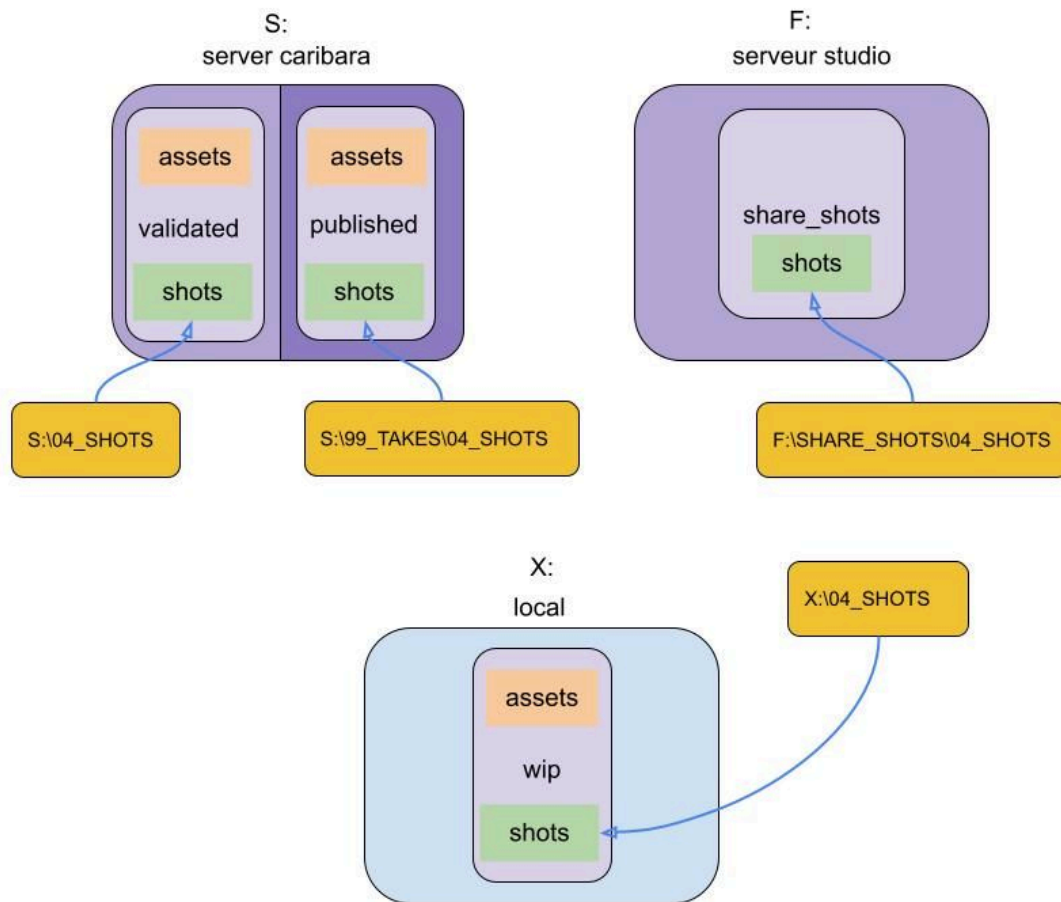
S contains all the files presented to the director as well as the associated export files and videos.

S also contains a "99_TAKES" subdirectory which forms a fourth area, containing the production database, i.e. any asset published to be used in a shot, or any shot published to be rendered in Unity. In the end, this directory will contain all the data necessary to reproduce any shot of any episode.

so we find here:

- on Caribara's synchronized server, in:
 - (S:\\ 99_TAKES\\03_ASSETS) all versioned assets as well as materials, textures and files needed for assets, and bg setdressing files.
 - ((S:\\ 03_ASSETS) the modeling/texturing/rigging files submitted to the director for validation.
 - ((S:\\ 04_SHOTS) layout / anim files subject to validation by the director.
 - (S:\\ 99_TAKES\\04_SHOTS) exports of approved shots (layout / anim) .
- on the server of each studio:
 - (F:\\SHARE_SHOTS) the files exchanged between the supervisor and the artists (layout et anim)
- on each operator's workstation :
 - the files he is working on and his personal iterations. ((X:\\ 03_ASSETS ou X:\\ 04_SHOTS)

Every location for the shots and their adresses



l'arborescence - tree structure [↗](#)

au sein de ces espaces, les arborescences sont exactement les mêmes:

within these area, the trees are exactly the same:

pour les assets: [↗](#)

ils sont tous dans un répertoire "03_ASSETS" qui contient les sous répertoires suivants:

- 01_CHARACTER
- 02_BACKGROUND
- 03_PROP
- 04_BGELEMENTS
- 05_FX
 - 08_BANK

On trouve à la racine de chaque répertoire, les .blend, et dans un sous répertoire "export" les fichiers fbx et mbx.

à noter:

- Les personnages possèdent en plus un répertoire à leur nom dans 01_CHARACTER contenant leurs fichiers de config.
- Le répertoire 08_BANK servira uniquement pour les fichiers d'animés des animateurs.

assets:

they are all in a "03_ASSETS" directory which contains the following subdirectories:

- 01_CHARACTER
- 02_BACKGROUND
- 03_PROP
- 04_BGELEMENTS
- 05_FX
 - 08_BANK

We find at the root of each directory, the .blend, and in a sub-directory "export" the fbx and mbx files.

to be noted:

- The characters also have a directory in their name in 01_CHARACTER containing their config files.
- The 08_BANK directory will only be used for animator animation files.

pour les shots: ↻

ils sont tous dans un répertoire "04_SHOTS".

Celui-ci contient autant de répertoire que d'épisodes, nommés ainsi: code-production + numéro (ex: SMJ101).

à l'intérieur un répertoire par séquence (nommés s1, s2...) et encore à l'intérieur un répertoire par shots (nommés p0010 pour le premier plan, p0020 pour le second. le cas échéant un shot d'insert pourrait se nommer p0011)

ces répertoires ne contiennent que des fichiers sbx, et cbx (et mbx accessoirement).

En plus de ces répertoires, on trouve dans 04_SHOTS un répertoire 01_LAYOUT et 02_ANIMATION contenant les fichiers blender originaux rangés par épisode.

à noter:

Ces fichiers blender n'existent pas sur S:99_TAKES.

Shots: ↻

They are all in a "04_SHOTS" directory.

This contains as many repertoires as there are episodes, named as follows: production's code + number (like SMJ101).

Inside there is a directory by sequence (named s1, s2...) and still inside a directory by shots (named p0010 for the first shot, p0020 for the second. If necessary an insert shot could be named p0011)

These directories contain only sbx and cbx (and mbx incidentally) files.

In addition to these directories, we find in 04_SHOTS a directory 01_LAYOUT and a directory 02_ANIMATION containing the original blender files arranged by episode.

to be noted:

These blender files do not exist on S:99_TAKES.

Backup - Backup ↻

2 types de versionning sont utilisés. L'objectif est toujours qu'à un fichier donné ne corresponde qu'un seul nom. 2 types of versioning are used. The objective is always that a given file corresponds to only one name.

- SVN:

- obsolete: on utilise pour les assets le meme principe de repo que pour les shots decrit plus bas (avec un repertoire de partage "share_assets").
- Pour les assets en cours de fabrication, nous utilisons SVN. c'est une database déposéée sur un serveur qui versionne les sauvegardes successives. L'utilisateur peut mettre ses fichiers à jour à partir de la database(update) et mettre la database à jour à partir de ses fichiers (commit) .
- on accède par défaut à la dernière version du fichier mais on peut comparer et récupérer une ancienne version si nécessaire.
- REPO:
 - Les shots sont versionnés sur le même principe mais en local sur le poste ou ils sont conçus. à chaque sauvegarde par les outils Manette, les versions antérieures sont copiées/renommées dans un sous repertoire backup.
 - Lorsqu'ils sont déposés sur S, un système de backup type synology prend le relais.
 - le partage de fichier se fait lui via un repertoire "share_shots" accessible a tous par les outils manette.
- SVN:
 - For in-process assets, we use SVN. It's a database deposited on a server which versions successive backups. The user can update his files from the database (update) and update the database from his files (commit).
 - The latest version of the file is accessed by default, but an older version can be compared and retrieved if necessary.
- REPO:
 - The shots are versioned on the same principle but locally on the workstation where they are designed. At each save with the Manette Tools, the previous versions are copied/renamed in a backup sub-directory.
 - When they are uploaded on S, a synology-type backup system takes over.

Versionning: ↻

- lorsque qu'un asset ou un shot est soumis à validation auprès du réalisateur, les fichiers déposés sur S (et indiqué dans shotgun) portent leur nom complété de la task à valider et de son numéro d'itération (exemples: *SMJ106_p0150_animation-2.blend*, *be_armchairB_rigging-1.blend*). tous les fichiers associés (sbx,cbx,mbx,mov) auront le même suffixe.
- Lorsqu'ils sont validés, les datas de la dernière itération soumise sont copiés dans la database (S:99_TAKES) et le suffixe task+iteration est remplacé par un numéro de version (v1 par défaut).

Versioning: ↻

- When an asset or a shot is submitted for approval by the director, the files uploaded on S (and indicated in shotgun) take their name completed with the task to validate and its iteration number (for instance: *SMJ106_p0150_animation-2.blend*, *be_armchairB_rigging-1.blend*). All associated files (sbx,cbx,mbx,mov) will have the same suffix.
- When they are approved, the data of the last submitted iteration are copied into the database (S:99_TAKES) and the task+iteration suffix is replaced by a version number (v1 by default).

Fichiers de sauvegarde et d'exports:

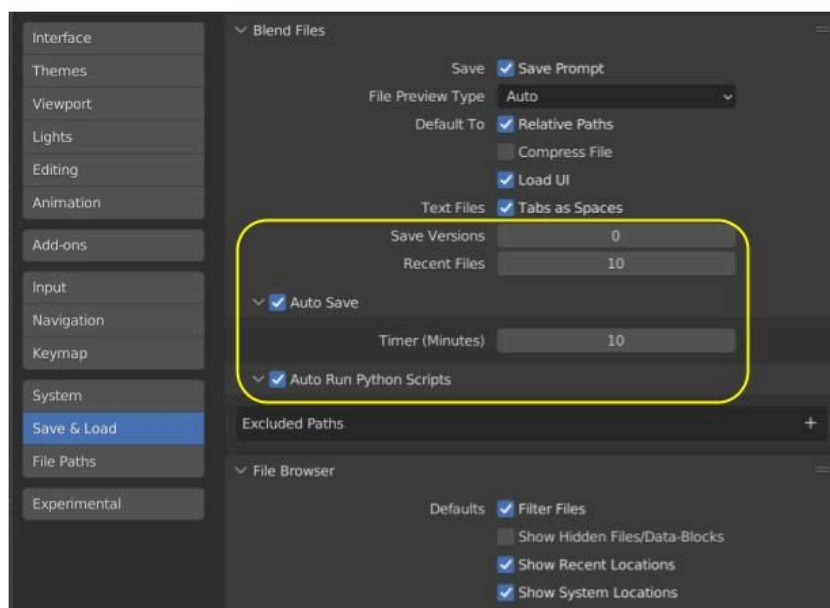
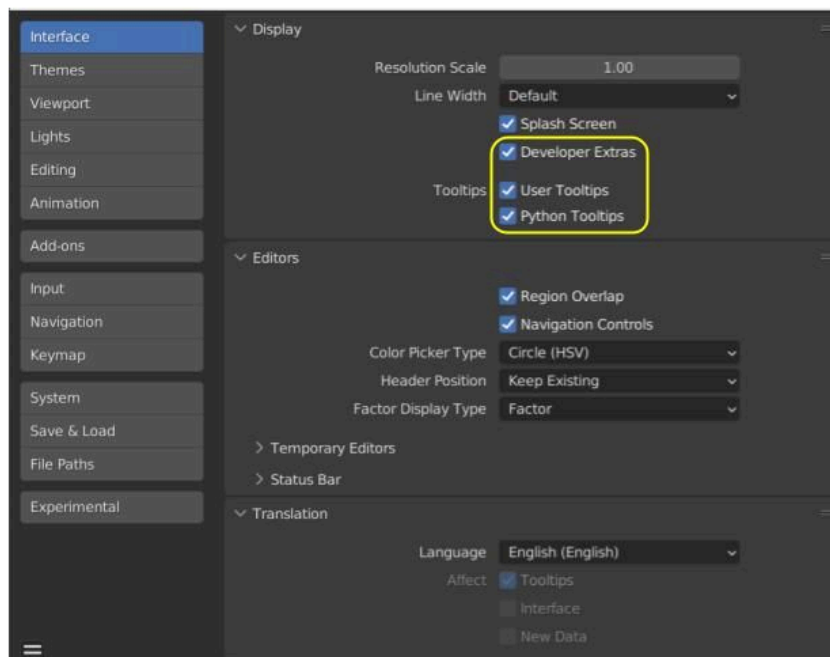
- Pour pouvoir recréer un shot dans blender comme dans Unity, les datas sont sauvegardées dans plusieurs fichiers d'exports.
 - la géométrie riggée et texturée est exportée en ***.fbx**
 - les paramètres de matériaux sont exportés en ***.mbx**
 - La configuration de la worldmap sera exportée au format **.wbx** et sera commune à toute la séquence (position des roots bg dans l'espace). Les worldmaps de l'épisode sont sauvegardées à la racine de la séquence.
 - le setdressing sera exporté en **.sbx** (position des assets vs le root du bg et info duration)
 - les animations seront exportées en **.cbx**
 - les matériaux seront exportés en **.mbx**
 - les assets riggés existent aussi en version .blend pour être animés dans blender.

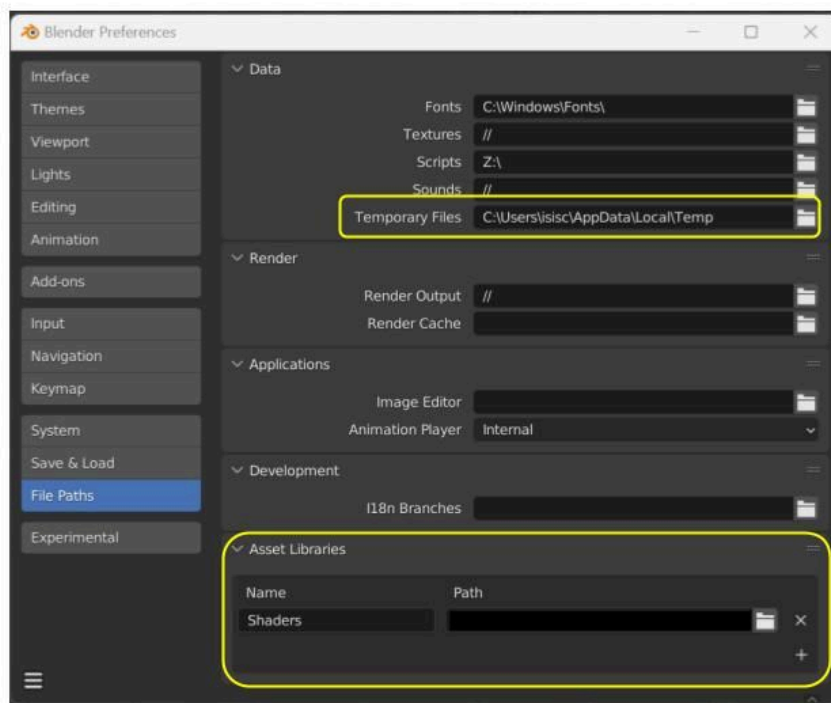
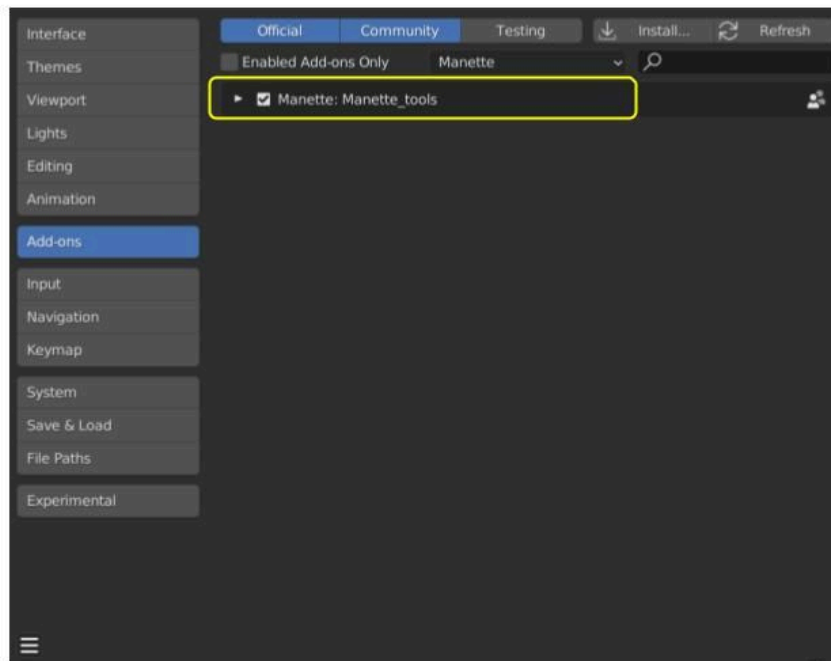
Backup and export files:

- To be able to recreate a shot in blender as in Unity, the data is saved in several export files :
 - rigged and textured geometry is exported as ***.fbx**
 - material settings are exported as ***.mbx**
 - The configuration of the worldmap will be exported in **.wbx** format and will be common to the whole sequence (position of the bg roots in space). The episode worldmaps are saved at the root of the sequence.
 - the setdressing will be exported in **.sbx** (position of the assets vs the root of the bg and info duration)
 - animations will be exported in **.cbx**
 - materials will be exported as **.mbx**
 - rigged assets also exist in .blend version to be animated in blender.

configuration blender - blender config

Une fois l'installation faite, à la première ouverture de blender il faut le configurer. cela se fait dans le menu Edit / préférences de blender. Once the installation is done, the first time you open blender, you have to configure it. This is done in the Edit / preferences menu of blender.





- Pour pouvoir visualiser correctement les propriétés contenues dans les assets Manette, les options Developers extras, users tooltips et python tooltips sont nécessaires.
- le système de versionning (save versions) de blender étant en double avec celui de Manette, il est désactivé (save versions = 0). Par contre, il est conseillé d'activer l'autosave (les sauvegardes automatiques sont accessibles via le menu File / Recover / autosave de Blender).
- Enfin il faut activer l'addons Manette_tools.

- To see correctly the properties contained in the Manette's asset, the options Developers extras, users tooltips et python tooltips are necessary.
- The versioning system (save versions) of blender being duplicated with the one of Manette, it is deactivated (save versions = 0). However, it is recommended to activate the autosave (the automatic saves are accessible via the Menu : File / Recover / autosave of Blender).
- Finally, you must activate the Manette_tools addons and restart blender.

Nomenclature et fichiers - Nomenclature and files: [↗](#)

la nomenclature - nomenclature [↗](#)

La nomenclature est définie dans un fichier de config. Elle peut être adaptée dans une certaine mesure. The nomenclature is defined in a config file. It can be adapted to some extent.

- **principes - principles**
- Tout nom dans un fichier est composé d'au moins trois mots : un nom unique, un "type" et une "tâche".
 - Le "type" est un mot-clé pour identifier le type d'asset (props, bg-parts, bg-elements, characters,...)
 - La "tâche" est un mot-clé pour identifier la fonction de l'objet composant l'asset (geometry, controller, master,...)
- Any name in a file is made up of at least three words: a unique name, a "type" and a "task".
- The "type" is a keyword to identify the asset type (props, bg-parts, bg-elements, characters,...)
- The "task" is a keyword to identify the function of the object composing the asset (geometry, controller, master,...)
- Chaque mot est séparé par le même caractère (" _ ").
 - Aucun caractère spécial sauf le séparateur.
 - Aucun chiffre dans le nom (dans les scènes, des numéros sont automatiquement rajoutés pour identifier les copies)
 - aucun espace (" ").
- Each word is separated by the same character (" _ ").
 - No special characters except the separator.
 - No numbers in the name (in the scenes, numbers are automatically added to identify the copies)
 - no space (" ").
- l'ordre des mots est: **type + nom1 + tache + nom2 + extras** The word order is: **type + name1 + task + name2 + extras**
- Les majuscules peuvent être utilisées mais ne sont pas nécessaires. Capital letters can be used but are not necessary.
- Pour les assets composés de plusieurs géométries, chacun a le nom de l'asset plus son propre nom. For assets consisting of multiple geometries, each has the name of the asset plus its own name.
- Pour les assets "bg", tous les composants portent le nom de l'asset suivi de leur propre nom. For "bg" assets, all components have the name of the asset followed by their own name.

- N'utilisez pas un mot plusieurs fois dans un même nom. (ex : "be_bluetable_md1_table") Do not use a word more than once in the same name. (eg: "be_bluetable_md1_table")
- avec ou sans majuscule ce n'est pas le même mot. With or without capitals it is not the same word.
- éviter les noms trop longs (un nom propre ne devrait pas dépasser 15 lettres). Avoid names that are too long (a proper name should not exceed 15 letters).
- L'outil de clean est utilisé pour adapter un nom donné à la convention de nommage de la production. The clean tool is used to adapt a given name to the production naming convention.
- Les informations supplémentaires (au-delà du nom, type et tâche) facultatives peuvent être composées de plusieurs mots séparés par un séparateur. Elles peuvent être utiles pour distinguer deux matériaux au sein d'un même mesh. Optional additional information (beyond name, type and task) can have several words separated by a "_". They can be useful to distinguish two materials within the same mesh.

• **mots clés - key words**

La liste ci-dessous de mots clés utilisés dans la nomenclature et leurs correspondances est celle de la production Sam et Julia. The list below of key words used in the nomenclature and their correspondence is that of the Sam and Julia production.

- pr = props
- bg = background
- be = bg éléments
- ch = character
- mdl = modelling
- shd = shader
- rig = rigging
- ctr = controller
- mstr = master

• **nomenclature des fichiers d'assets - nomenclature of asset files:**

- les fichiers .blend (ainsi que les fichiers sbx et mbx décrivant les bg) portent le nom "sans numéro" de l'asset. The .blend files (as well as the sbx and mbx files describing the bg files) has the name "without number" of the asset. ex:
 - ch_julia.blend
 - bg_mmPatio.blend
 - bg_mmPatio.sbx
 - pr_ballTennis.blend
 - be_armchairA.blend
- les fichiers fbx et mbx portent le nom de l'asset suivi de "01". fbx and mbx files have the asset name followed by "01". Ex :
 - ch_julia01.fbx
 - bg_mmPatio01_windowA01.fbx
 - pr_ballTennis01.fbx

- be_armchairA01.fbx

- **nomenclature fichiers de shot - nomenclature shot files:**

les fichiers de shots s'écrivent **code-production + num-episode + num-shot + task**exemple:

SMJ101_p0010_animation.blend (plan 1 de l'épisode 1)remarque: dans sam et julia, il n'y a pas de numero de sequence inclus dans le nom.shot files are written code-production + num-episode + num-shot + task

example: SMJ101_p0010_animation.blend (shot 1 of episode 1)

note: in sam and julia, there is no sequence number included in the name.

- **nomenclature dans les scènes de bg:**

Dans les fichiers les objets qui composent l'asset se nomment ainsi:Le nombre de champs composant un nom d'objet est généralement de 4.

In the files, the objects that make up the asset are named as such: The number of fields in an object name is usually 4.

- le premier champ est toujours occupé par le type (bg, ch, pr, fx)
- le deuxième champ est toujours occupé par le nom de l'asset
- le troisième champ décrit la fonction de l'objet (mdl,ctr,bon,hlp,...)
- Le quatrième champ est le nom complémentaire. Si c'est un élément de bg, c'est le nom de l'asset. si c'est un props, c'est le nom de la sous-partie, ou à défaut un mot générique ("root","body","base",etc...). Il n'est indispensable que pour les objets "bg".
-
- the first field is always occupied by the type (bg, ch, pr, fx)
- the second field is always occupied by the name of the asset
- the third field describes the function of the object (mdl,ctr,bon,hlp,...)
- The fourth field is the additional name. If it is a bg element, it is the name of the asset. If it is a props, it is the name of the subpart, or failing that a generic word ("root", "body", "base", etc.). It is only required for "bg" objects.
-

Chaque objet voit ses noms complétés d' un numero ("01")

Exemples:

- bg_hseRonchon01_mdl_wallL01
- be_tableB01_ctr_table01_mstr
- pr_doorHseRonchonF01_rig

Each object has its name completed with a number ("01")

Examples:

- bg_hseRonchon01_mdl_wallL01
- be_tableB01_ctr_table01_mstr
- pr_doorHseRonchonF01_rig

- **nomenclature dans les scènes de shots:**

Dans un shot, plusieurs bg peuvent cohabiter, et ainsi partager des copies adaptées d'un même bg-élément original.

pour conserver le lien entre un bg et les assets qui le composent dans un shot, le nom des bg-éléments se voit complété d'un préfixe [ID du bg] devant son nom

exemple:

- be_[JUL01]nightstandShelfA01_ctr_shelf01_mstr

- **nomenclature in shot scenes:**

In a shot, several bg's can coexist, and thus share adapted copies of the same original bg-element. To keep the link between a bg and the assets that compose it in a shot, the name of the bg-elements is completed with a prefix [bg ID] before its name

exemple:

- be_[JUL01]nightstandShelfA01_ctr_shelf01_mstr

- **nomenclatures matériaux:**

Les noms des matériaux découlent des noms des assets. le type devient ["shd"]

exemples:

- ch_tchouchouka01_shd_boots01
- bg_school01_shd_birchTree01_Leaves
-
- material nomenclature:

The material names are derived from the asset names. the type becomes ["shd"]

examples:

- ch_tchouchouka01_shd_boots01
- bg_school01_shd_birchTree01_Leaves

- **nomenclature textures:**

les noms des textures découlent des noms des assets:

peut suivre aussi le nom de la sous-partie de l'asset concernée

ex: bg_school01_bigTree01_leaves_diff

la caractéristique de la texture (diffuse,etc...) se place en dernier.

le type ["mdl/shd/rig"] est retiré

suffixes possibles:

- STICKER (Couleur + Alpha si besoin)
- LN (luminance + Normal map)

- SM (splatmap)
- ALBEDO (couleur + Alpha si besoin)
- MASKMAP (Metalness,Smoothness,AO)

- **texture nomenclature:**

Texture names are derived from asset names:

can also follow the name of the relevant asset subpart

ex: bg_school01_bigTree01_leaves_diff

the texture characteristic (diffuse, etc...) is placed last.

type ["mdl/shd/rig"] is removed

possible suffixes:

- STICKER (Colour + Alpha if needed)
- LN (luminance + Normal map)
- SM (splatmap)
- ALBEDO (Colour + Alpha if needed)
- MASKMAP (Metalness,Smoothness,AO)

- **nomenclature collections:**

Les assets sont regroupés au sein de collections. La gestion des collections et la création / gestion des controller étant scriptée, il n'y a pas lieu de s'en soucier à priori.

On trouvera toutefois le détail du fonctionnement [ici](#).

si nécessaire, l'outil de clean permet d'organiser automatiquement les collections.

- **nomenclature collections:**

Assets are grouped into collections. As the management of collections and the creation/management of controllers is scripted, there is no need to worry about it.

However, the details of how this works can be found [here](#).

if necessary, the clean tool can automatically organise the collections.

- **nomenclature rig:**

on utilisera la nomenclature de rigify.

les règles sont:

- Il y a deux types de noms de bones: ceux avec préfixe et ceux sans.
- Les préfixes peuvent être: ORG,DEF,MCH. ils sont toujours suivis d'un tiret ("-").
- tous les bones déformant ont le préfixe DEF.
- Les différentes parties d'un mots sont séparées par un underscore ("_").

- Il y a deux types de suffixes: ceux décrivant la latéralité (L,R) en majuscule, et des numéros (001,002). Les suffixes sont toujours précédés d'un point.

exemple: DEF-upper_arm.L.001

pour les assets sans rig (tous les éléments de décor statiques)

tous ont au moins un controller racine (master) portant le suffixe “_mstr” et possiblement un ou plusieurs sous controllers enfants pour manipuler des parties distinctes.

Ces controller sont des empty créés et nommés automatiquement à l'aide des outils.

- **nomenclature rig:**

Rigify nomenclature will be used.

The rules are:

- There are two types of bone names: those with prefixes and those without.
- The prefixes can be: ORG,DEF,MCH. they are always followed by a dash ("-").
- all deforming bones have the prefix DEF.
- The different parts of a word are separated by an underscore ("_").
- There are two types of suffixes: those describing laterality (L,R) in upper case, and numbers (001,002). Suffixes are always preceded by a dot.

exemple: DEF-upper_arm.L.001

for assets without rig (all static set elements)

All have at least one root controller (master) with the suffix "_mstr" and possibly one or more child sub-controllers to handle separate parts.

These controllers are automatically created and named by the tools.

process comuns (common processes): ↻

Le shot est créé par assemblage dans blender des assets contenus dans la base caribara. Lorsque la tâche layout ou anim est validée par le superviseur, elle peut être envoyée (**SEND**) pour soumission au réalisateur. Pour ce faire,une copie sera déposée sur le serveur caribara et les liens vers les fichiers transmis via shotgun. Si le shot est entièrement validé (layout + anim) une copie des fichiers d’export sera publiée.

:The shot is created by assembling in blender the assets contained in the caribara database. When the layout or anim job is validated by the supervisor, it can be sent (SEND) for submission to the director. To do this, a copy will be uploaded to the caribara server and the links to the files transmitted via shotgun. If the shot is fully validated (layout + anim) a copy of the export files will be published.

exemples de sauvegarde de fichiers de travail

OS (X:) > 04_SHOTS > 02_ANIMATION > SMJ128

Nom

 SMJ128_p0360_animation.blend

OS (X:) > 04_SHOTS > 01_LAYOUT > SMJ128

Nom

 SMJ128_p0360_layout.blend

 > Ce PC > OS (X:) > 04_SHOTS > SMJ106 > s1 > p0050		
Nom	Taille	Type
 SMJ106_p0050.mbx	370 Ko	Fichier MBX
 SMJ106_p0050.sbx	15 Ko	Fichier SBX
 SMJ106_p0050_ch_julia01_rig.cbx	2 092 Ko	Fichier CBX
 SMJ106_p0050_ch_julia01_rig_layout.cbx	295 Ko	Fichier CBX
 SMJ106_p0050_ch_sam01_rig.cbx	3 171 Ko	Fichier CBX
 SMJ106_p0050_ch_sam01_rig_layout.cbx	252 Ko	Fichier CBX
 SMJ106_p0050_pr_cam01_rig.cbx	3 Ko	Fichier CBX
 SMJ106_p0050_pr_cam01_rig_layout.cbx	6 Ko	Fichier CBX

 > Ce PC > OS (X:) > 04_SHOTS > SMJ106 > s1		
Nom	Taille	Type
 p0010		Dossier de fichiers
 p0020		Dossier de fichiers
 p0030		Dossier de fichiers
 p0040		Dossier de fichiers
 p0050		Dossier de fichiers
 SMJ106_s1.wbx	22 Ko	Fichier WBX

exemples de sauvegarde pour l'export

à noter:

Il n'est pas prévu que l'on exporte de la géométrie en layout ou en anim.

le cas échéant les exports se feraient à l'aide des tools Manette aux formats .fbx ou .blend.

to note:

There are no plans to export geometry to layout or anim.

In the event that it is, exports would be made using the Manette tools in .fbx or .blend formats.

tasks:

Les tâches assignées à un opérateur via shotgun sont accessibles depuis les tools manette. Pour un certain nombre d'outils, il est nécessaire d'avoir une tâche de sélectionnée pour qu'ils s'exécutent correctement.

Pour associer un fichier blender à une tâche, on peut soit sélectionner une tâche et ouvrir le fichier correspondant (si il n'existe pas il sera créé un fichier vide), soit ouvrir un fichier et sélectionner ensuite la tâche correspondante. à la sauvegarde, le fichier prendra le nom de la tâche.

tasks:

The tasks assigned to an operator via shotgun are accessible from the joystick tools. For a number of tools it is necessary to have a selected task for them to run correctly.

To associate a blender file with a task, you can either select a task and open the corresponding file (if it does not exist, an empty file will be created), or open a file and then select the corresponding task. when saving, the file will take the name of the task.

exports manuels:

tous les fichiers d'exports sont produits automatiquement lors d'un renvoi de tâche (**SEND**). mais on peut aussi produire ces fichiers indépendamment et à tout moment. ils peuvent être soit exportés avec les chemins et noms par défaut (ceux correspondant à la tâche) ou ailleurs.

manual exports:

All export files are automatically produced when a task is returned (SEND). But these files can also be produced independently at any time. they can either be exported with the default paths and names (those corresponding to the task) or elsewhere.

publishing:

Le publishing des assets peut se faire individuellement ou par bg, sauf pour les rig qui sont publiés par lot séparément. On peut toujours publier n'importe quel asset individuellement.

Le publishing des shots se fait par lot après validation de l'anim. Les dernières itérations postées à destination de la production. sont copiées sur le serveur à destination du rendu.

publishing:

Asset publishing is done by bg or individually, except for rigs which are published by batch separately. You can always publish any asset individually.

Shots are published in batches after validation of the anim. The last iterations posted to production are copied to the server for rendering.

première utilisation / changement de production: ↻

- Après avoir ouvert Blender, ouvrir directement l'interface Manette, sans ouvrir de fichier particulier.
- Commencez par vous logger sur aquarium pour avoir accès à une production et les tâches qui vous sont assignées.
- Ensuite choisissez une production.
- Enfin cliquez sur "set default log"

ce faisant vous avez stocké en local vos log et la production sur laquelle vous travaillez.

A l'avenir, vous pourrez directement sélectionner une tâche, sans avoir besoin de refaire les étapes de log / production / set default log.

Si vous deviez plus tard changer de production, vous aurez à refaire les étapes production / set default log.

first use / change of production:

- After opening Blender, open the ManetteUI interface directly, without opening any particular file.
- Start by logging into aquarium to access a production and the tasks assigned to you.
- Then choose a production.
- Finally click on "set default log".

By doing this you have stored your logs and the production you are working on locally.

In the future you will be able to select a job directly, without having to go through the log / production / set default log steps again.

If you were to change production later, you would have to redo the production / set default log steps.

tools communs: ↻

- **Infos Panel**
 - contient des infos produites par les tools.

- **preset Panel:**
 - **selection of UI:**

détermine les tools disponibles dans Tools Panel.

determines the tools available in the Tools Panel.

- **Lod:**

détermine le niveau de détail des assets affichés dans la scène.

determines the level of detail of the assets displayed in the scene.

- **preset of textures size:**

détermine la taille des textures affichées dans la scène.

determines the size of the textures displayed in the scene.

- **force textures conform:**

recharge toutes les textures de la scène à la taille choisie.

reloads all the textures in the scene at the chosen size.

- **set default UI:**

sauvegarde les paramètres de presets de l'utilisateur.

saves the user's preset settings.

- **Log Panel:**

- **login:**

adresse email et mot de passe nécessaires pour se logger à aquarium.

email address and password needed to log in to aquarium.

- **production:**

liste des productions auxquelles l'utilisateur à accès.

list of productions to which the user has access.

- **set default log:**

sauvegarde le log/mot de passe et la production de l'utilisateur.

à noter: une fois fois sauvegardé, il n'est plus nécessaire de se logger,

ou de sélectionner sa production au lancement de blender.

saves the user's log/password and production.

note: once saved, it is no longer necessary to log in,

or to select your production when launching blender.

- **Task Panel:**

- **filters tasks:**

(supervisor tool) Filtre l'ensemble des tâches disponibles sur shotgun, par tâche et par statut. (Le résultat est visible dans les **tasks**.)

(supervisor tool) Filters all available tasks on shotgun, by task and by status. (The result is visible in the tasks).

- **change file after submission:**

(supervisor tool) Permet de mettre à jour une des itérations préalablement déposées d'un fichier pour validation auprès de la production. (permet à un superviseur de corriger un envoi erroné)

(supervisor tool) Allows one of the previously submitted iterations of a file to be updated for validation by production. (allows a supervisor to correct an erroneous submission)

- **update tasks:**

Update la liste des tâches assignées à l'utilisateur. (Le résultat est visible dans les **tasks**.)

Update the list of tasks assigned to the user. (The result is visible in the tasks).

- **filtering:**

Filtre les tâches listées contenant un mot donné. (Le résultat est visible dans les **tasks**.)

Filter the listed tasks containing a given word (the result is visible in the tasks).

- **tasks:**

Liste des tâches disponibles. ([voir detail](#))

List of available tasks. ([see detail](#)).

- **OPEN:**

charge le fichier blender correspondant à la tâche sélectionnée ou à défaut une scène vide. ([voir detail](#))

loads the blender file corresponding to the selected task or an empty scene ([see detail](#)).

- **SHARE:**

partage le fichier de la scene courante dans le dossier dédié. ([voir detail](#))

share the current scene in the dedicated folder(see detail)

- **SAVE:**

sauvegarde la scène courante à partir du nom de la task sélectionnée. ([voir detail](#))

saves the current scene from the selected task name. (see detail)

- **ASK to REVIEW:**

permet de changer le statut de la tâche en cours en "wait for approval" pour signaler au superviseur que l'on attend son retour. le fichier est partagé via shotgrid et l'espace de partage (SHARE_SHOTS)

allows you to change the status of the current task to "wait for approval" to tell the supervisor that you are waiting for his return. the file is shared via shotgrid and the share space (SHARE_SHOTS)

- **create render target:**

(asset tools)

crée une cible (qui sert de bounding-box) pour forcer la caméra de **RENDER** à se concentrer sur une partie de la scène lors d'un shoot d'asset.

creates a target (which serves as a bounding box) to force the RENDER camera to focus on one part of the scene during an asset shoot.

- **RENDER:**

fait un rendu qui pourra être mis à disposition sur shotgun.

En fonction de la tâche en cours, il générera la caméra et les lumières pour un turnaround (assets), ou utilisera ceux présents dans la scène (shots).

à lancer au préalable d'un Ask To Review ou d'un SEND.

makes a render that can be made available on shotgun.

Depending on the current task, it will generate the camera and lights for a turnaround (assets), or use those present in the scene (shots).

To be launched before an Ask To Review or a SEND.

- **SEND:**

renvoi une tâche terminée via shotgun([voir detail](#)).

- change de statut sur shotgun.
- dépose les rendus correspondant s'il en trouve.
- sauvegarde la scène (tel que SAVE)
- exporte la scène (sbx et cbx)
- copie le fichier blender et les fichiers d'exports correspondants sur le serveur caribara.

return a completed task via shotgun.

- changes the status on shotgun.
- drops the corresponding renderings if it finds any.
- saves the scene (as SAVE)
- exports the scene (sbx and cbx)
- copy the blender file and the corresponding export files on the caribara server.

- **Clean Panel:**

- **sanity check:**

[0.9.5][en cours de réécriture]

Vérifie un ensemble de propriété des assets (Ngon,nomenclature,etc) mais ne modifie rien.

paramétrable avec le fichier BlenderTools\qtutils\prodSanity.json

sert à verifier un ensemble d'erreurs possibles et d'infos,dont:

- la nomenclature et l'organisation des collections et objets.
- zero transform et lock transform
- erreurs de topologies
- nombre de triangles/polygones
- nomenclature et chemins de textures/materials

Checks a set of asset properties (Ngon,nomenclature,etc) but does not change anything.

- **Clean scene:**

Conforme les assets (nomenclature, transform,collection..)

Conforms assets (nomenclature, transform, collection..)

la nomenclature est decrite dans le fichier BlenderTools\qtutils\prodDef.json

- **Clean shot:**

Conforme les shots (nomenclature, transform,collection..)

Conforms to the shots (nomenclature, transform, collection..)

- **Export Panel:**

- **Custom path:**

(on/off) Utilisation du custom path à la place des chemins par défaut pour tous les exports manuels.

- **Custom path and name:**

sélection du chemin et nom du fichier du custom path. (sans suffixe .#bx)

- **Export custom setdress:**

Exporte du set-dress de la scène courante au format .sbx

- **import custom setdress:**

Importe d'un set-dress au format .sbx

- **export rigged asset:**

Exporte d'un asset riggé sous la forme de 2 fichiers du même nom: un .fbx (modé et rig) et un .mbx (matériaux)

- **export no rig asset:**

Exporte d'un asset sans rig (parenté à un empty "MASTER") sous la forme de 2 fichiers du même nom: un .fbx (modé et controllers) et un .mbx (matériaux)

- **cbx export deformers anim:**

Exporte l'animation des bones "déformer" d'un rig dans un fichier .cbx. s' ils n'ont pas de clés (comme sur un personnage), les clés seront "projetées".

utilisable dans unity (et avec des characters "crowd", dans blender et unity)

- **cbx export controllers anim:**

Exporte l'animation des bones "controller" d'un rig dans un fichier .cbx.

utilisable dans blender (et avec des rig de props simples, dans blender et unity)

- **cbx import anim:**

import l'animation incluse dans un fichier cbx et l'assigne au personnage sélectionné.

un fichier cbx controllers est taggé "ANI" sur sa première ligne, et "CHR" pour un fichier cbx deformers

- **export mbx:**

Exporte uniquement les matériaux de l'asset sélectionné dans un .mbx

- **export fbx:**

Exporte uniquement l'asset sélectionné dans un .fbx (sans les matériaux)

process de fabrication des assets: ↻

pré modeling: ↻

- En partant d'un fichier blender vide, on sélectionne la tâche dans les ManetteTools.
- on fait une greybox ou un premier modeling de façon à pouvoir déterminer son point de pivot.
- on lui donne un nom minimum (ballon).
- on ajuste son point de pivot.
- on lui crée un master avec le tool add master for selected asset.
- on sélectionne le master et on le conforme avec l'outil clean scène (toutes les options actives).

(adaptation du nom, des collections, du nom des matériaux, et des transforms)

- on vérifie que la nomenclature est celle souhaitée
- on refait un clean scene complet.
- on le sauvegarde avec le tool save des manetteTools. (en local sur X)

review sup: ↻

- Pour soumettre son asset au superviseur, on utilise le tool ask to review.

(le statut est changé et le fichier est copié sur share.)

- une fois l'asset validé par le superviseur il peut être soumis au réalisateur avec le tool send.

(le statut change, et le fichier est copié sur share et sur S renommé avec sa tâche.

vie de l'asset: ↻

pour les étapes suivantes (uv,etc..), on repart du fichier disponible sur share (sauf si on est l'auteur de la tâche précédente):

- sélectionner la tâche.
- ouvrir le fichier correspondant avec open (option “from_share”)

(il n’y a qu’un seul fichier pour toutes les tâches)

- Le fichier est copié en local et ouvert ensuite.
- et le même processus peut continuer.

tools de modeling:

Tools Panel — modeling tools

add simple master for all bg assets

ajoute un master a tout les elements de bg de la scene. ([voir details](#))

add master for selected assets

permet de placer un ou plusieurs sous controllers en meme temps que le master. ([voir details](#))

merge character model

importe un personnage en supprimant son rig et en le remplaçant par un master, en vue de s’en servir de ref size.

convert to prop

permet de replacer la hierarchie de master et sous controller d’un asset par un rig equivalent

clone selected assets

permet de cloner un asset en le renommant / rangeant correctement

switch to rigged asset

permet de remplacer l’asset selectionné par sa version “props” si elle existe

force confo

permet d’associer à l’assets un certain nombre de propriétés decrivant sa nomenclature et sa fonction.

remove assets

permet de supprimer definitivement un asset et toutes ses dependances

replace bg data copies

permet de remplacer la geometrie / materiaux d’un asset dans toutes ces copies dans la scene.

Tools Panel — BG tools

push blendshape on original [↗](#)

open original asset [↗](#)

build BG from BGelements [↗](#)

Tools Panel — proxy tools [↗](#)

replace geo by primitive [↗](#)

replace geo by selection [↗](#)

replace geo by decimate [↗](#)

replace all geo by decimate [↗](#)

convert asset to proxy [↗](#)

Tools Panel — combo tools [↗](#)

create combo asset [↗](#)

update combo asset [↗](#)

open combo asset [↗](#)

process de layout: [↗](#)

le process de layout se déroule comme suit:

Dans un premier temps, on sélectionne une tâche de layout avec **TASK**.

Puis avec la fonction **populate** on crée un nouveau fichier avec tous les éléments nécessaires au shot ainsi qu'une caméra riggée. Les différents bg qui composent le shot seront automatiquement assemblés selon la worldmap correspondante. Ceux n'existant pas dans la worldmap seront à connecter ou placer.

s'en suivent les étapes classiques de layout: adaptation des décors, animation de la caméra, et anim "démon" des personnages nécessaire à la validation des cadrages.

Une fois la mise en scène terminée, le fichier blender est sauvegardé et ses composantes exportées.

les animations seront exportées avec le suffixe "_layout.cbx"

- set-dressing:
 - On ne manipule jamais la géométrie, seulement les empty (masters ou controllers) ou les bones.
 - On anime jamais les empty, uniquement les bones.
 - Le setdressing est sauvegardé au format sbx et inclut position/rotation/échelle et hiérarchie de tout empty dans la scène.
 - pour dupliquer un asset, utiliser le tool manette dédiée.
- merge d'un asset non fourni:
 - On peut merger n'importe quel asset depuis la database.
- suppression d'éléments:
 - utiliser l'outil remove asset.
- duplication d'un élément:
 - utiliser l'outil clone asset
- optimisation de la scène:
 - pour l'optimisation du viewport, masquer les éléments hors champs, utiliser le mode shading et paramétrer le LOD en mid ou low.
 - pour l'optimisation des rig, utiliser les versions LOD rig si elle existe.
- animation de la caméra:
 - L'animation se fait en mode POSE du rig.
 - utiliser la caméra riggée intégrée à la scène. La focale (animée ou pas) sera exportée en même temps que l'animation.
- animation des personnages:
 - L'animation se fait en mode POSE du rig.
 - utiliser un des bones "global", "root" ou "move" pour le déplacement. vous pouvez utiliser l'interface d'anim pour sélectionner un bone facilement et utiliser la librairie de pose.
- animation des props:
 - L'animation se fait en mode POSE du rig.

- pas d'interface dédiée aux props.
 - voir techniques d'animation dans blender.
- animation d'autres éléments:
 - pas d'animation d'asset qui ne soit pas riggés avec des bones.
 - Si nécessaire, on peut demander la version riggée d'un élément de décor.
- connection de 2 bg:
 - Les bg sont composés, en plus des assets et du root, d'une ou plusieurs connections permettant de repérer le point de jonction de deux bg. Ces connexions sont représentées par un empty permettant de repérer l'intérieur de l'extérieur du bg. Il lui est associé un asset (porte,etc...). chaque connexion à son pendant dans un autre bg avec un asset identique (la même porte). L'un des deux exemplaires de l'asset sera supprimé au moment de la connexion. Les 2 connexions seront alignées (le "in" de l'une sur le "out" de l'autre).

tools de layout: ↻

- **Tools Panel:**
 - **populate:**
 - Le tool utilise la breakdown-list associé au shot dans shotgun pour peupler la scène. il merge les assets depuis le serveur caribara. Si il existe déjà une worldmap pour la séquence auquel le shot appartient, elle sera utilisée pour placer les bg. Sinon, en fonction des bg listés il sélectionne la worldmap générique correspondante.
 - il existe deux possibilités pour récupérer les assets:
 - à partir des **fichiers fbx** de chaque asset listés dans le sbx de chaque bg. (liste exhaustive, données 100% conformes, plus lent)
 - à partir des **fichiers blender** si ils sont accessibles.(potentiellement moins "propre", un fichier blender pouvant contenir des données non conformes, plus rapide). Cette option devrait disparaître.
 - Pour l'instant, on ne prévoit pas de mettre les fichiers .blend à disposition exception faite des assets riggés qui seront toujours des fichiers blend.
 - **open database:**
 - ouvre la database qui permet de chercher par mot clé un asset et de le merger dans la scène.
- **clone selected asset:**
 - sélectionner le master ou le rig d'un asset pour dupliquer l'asset correctement (nom, collection, nom du matériau)
- **create rigged camera:**
 - crée une caméra riggée. (chaque shot créé avec populate en contient déjà une.)
- **worldMap Panel:**
 - **select generic WM:**
 - permet de sélectionner une worldmap générique. (destiné au superviseur bg)

- **create new generic WM:**
 - permet de créer une nouvelle worldmap générique. (destiné au superviseur bg)
- **load WM:**
 - charge une worldmap et place les bg (soit celle de la séquence de la task en cours, soit celle du générique sélectionnée avec select generic WM).
- **save WM:**
 - sauvegarde la worldmap de la scène, soit comme celle de la séquence de la task en cours, soit comme générique sous le nom sélectionné.
- **show / show all / hide WM:**
 - show: affiche la worldmap, masque le reste.
 - show all: affiche tout.
 - hide WM: masque la worldmap, affiche le reste.
- **remove WM:**
 - supprime la worldmap (et les boundingbox qui vont avec) de la scène.
- **add node to world map:**
 - sélectionner un root de bg n'existant pas dans la worldmap. une nouvelle node au nom du bg est créée et ajoutée à la worldmap en cours (à sauvegarder avec save WM).
- **merge full bg:**
 - sélectionner une node de worldmap. le bg correspondant est mergé même s'il n'est pas listé dans la breakdown list.
- **connect 2 bg:**
 - sélectionner la connexion correspondant au bg à déplacer pour le connecter, puis la connexion à laquelle il doit être raccordé. Le premier bg sera aligné sur le second au point de connexion.

process d'anim:

Le fichier est reconstruit à partir des exports layout du shot (sbx,cbx,wbx,mbx) et des assets présents sur le serveur caribara.

À la fin du processus d'animation, ce sont ces mêmes fichiers d'exports qui seront fournis à Manette pour reproduire le plan.

tools d'anim:

- **Tools Panel:**
- **multi open:**
 - (supervisor tool) monte plusieurs shot layout pour être disponible à l'animation.
- **multi send:**
 - (supervisor tool) envoi plusieurs shots pour validation au réalisateur.
- **shot publish:**
 - (supervisor tool) publie plusieurs shots à destination du rendu.
- **open database anim:**
 - donne acces aux databases d'anim
- **interface anim:**
 - contient un picker qui s'adapte au personnage sélectionné.
- **update character list:**
 - force l'update de la liste des personnages présents dans la scène.
- **switch to LOD rig:**
 - remplace le personnage animé sélectionné par une version "low rig" tout en conservant l'animation. l'anim n'est pas modifiable en LOD rig.
 - processus un peu long, compter 1 minute / 200 frames environ.
- **switch to full rig:**
 - remplace le personnage low rig sélectionné par sa version "full rig" en conservant l'animation. l'anim est à nouveau modifiable.
- **update assets:**
 - permet d'updater un shot, à partir de la breakdown-list de shotgrid ou du sbx (si il à été mis à jour)
- **remove assets:**
 - supprime proprement un asset de la scene.
- **CROWD Panel:**
 - **open crowd-anim-builder:**
 - tool destiné à convertir les animations en crowd anim (utilisées pour les personnages non jouant)

- **merge crowd model:**
 - merge un character "crowd"
- **assign cbx crowd:**
 - assigne une animation à un character "crowd"
-

task panel:

tasks

point de depart de tout le processus de travail.

- Choisissez simplement la tache que vous souhaitez commencer ou poursuivre.

dans le task panel apparaitra son nom et son statut

remarque

de la meme façon qu'en selectionnant une production, vous parametrez les chemins d'accès aux fichiers en lecture comme ecriture, en selectionnant une tache, vous parametrez les outils pour qu'ils s'adaptent à ce que vous avez à faire: un asset ou un shot.

open

ouvre le fichier correspondant à la tache selectionnée.

si c'est un fichier de preproduction (modeling/.../rigging)

- si le fichier n'existe pas encore il sera crée à partir d'un fichier vide.

si c'est un fichier de production (layout / anim).

- Si le fichier n'existe pas encore, il sera crée, à partir des fichiers setdress/anim ou à défaut, de la breakdownlist.
- si il existe, il sera mis à jour à partir des setdress/anim contenus dans le repertoire dédié au plan correspondant.
- si un objet riggé est deja animé, son animation est conservée.
- si par contre il ne l'est pas dans la scene mais qu'il y a une animation à son nom presente sous forme d'un fichier cbx, il sera animé à partir des données de ce fichier.

share

- pour sauvegarder son fichier (en local) et le partager en le déposant sur share.

save

- Pour sauvegarder en local (sur X) avec le bon nom et au bon endroit (en fonction de la task selectionnée) quel que soit l'origine du fichier courant ouvert.

important:

le nom sous lequel le fichier est sauvegardé n'est pas le nom courant lisible dans l'angle haut/gauche de blender, mais le nom de la tache.

ainsi vous pouvez faire autant de sauvegarde par l'intermédiaire des raccourcis claviers habituels que vous le souhaitez, sous des noms différents (incrémentation, versionning, etc...), le nom de sauvegarde lorsque vous utilisez la fonction SAVE de l'interface manette est toujours celui de la tâche, et le répertoire toujours le répertoire dédié.

pour les fichiers de production (layout, anim), seuls les fichiers setdress et cbx sont commit sur SVN, les fichiers blender restent en local sur votre machine. si vous souhaitez les rendre accessibles à tous, vous pouvez aller dans le répertoire windows concerné et passer par le menu svn (bouton droit de la souris sur le fichier, add, puis commit) ou les déposer dans un répertoire dédié sur le repo.

render

- si vous souhaitez faire un rendu, en vue de l'envoyer à l'attention du lead/réa, cliquez sur RENDER. il sera stocké en attendant que vous validiez la tâche (SEND) et sera posé sur aquarium à ce moment là. Des précisions et des évolutions autour du photomaton (RENDER) suivront ultérieurement. n'hésitez pas à demander en attendant si nécessaire. vous trouverez les rendus dans un sous répertoire "\shoot" ex pour un bg: S:\01-PREPROD\03-BG\shoot

send

fonctionne avec les prod administrées avec aquarium.

- Quand la tâche est finie, cliquez sur SEND pour la sauvegarder et signaler que la tâche est disponible à la validation. si un rendu est a été fait pour cet asset ou ce shot, il sera envoyé sur aquarium et rajouté au media correspondant.

pour les productions entièrement sur aquarium:

pour les productions entièrement sur aquarium, les outils suivants existent

look for media and notes:

- ouvre une page web (ne fonctionne pas avec shotgun actuellement) vous permettant de voir directement les medias associés à la tâche sélectionnée (animatique, plan layout) et les notes, commentaires et échanges autour de cette tâche.

send timelog

fonctionne avec les prod administrées avec aquarium.

- En fin de journée, pensez à envoyer vos log. pour ce faire cliquez sur send timelog. Une liste de tâche apparaît (celles que vous avez lancées dans la journée) et vous devez simplement indiquer en face le nombre d'heures (approximatif) que vous y avez consacré ce jour.

change day

fonctionne avec les prod administrées avec aquarium.

- si vous avez oublié la veille (ou plus anciennement) d'envoyer vos log, vous les retrouverez en cliquant sur change day, et en choisissant parmi la liste, le log journalier que vous souhaitez envoyer (seul ceux en attente seront listés). après c'est pareil: send timelog.

modeling panel:

add master for all bg assets:

destiné uniquement aux elements de background ("bg_")

n'intervient que sur les objets "originaux" c'est à dire dont le 2eme champs serait "mdl" et 4eme champs se termine par "01"

- crée un master controller portant un nom adapté à l'objets (remplace "mdl" par "ctr" et ajouter le suffixe "mstr")
- reset le transform du ou des objets (mdl)
- aligne le master en position sur l'objet
- parente le ou les objets au master

add master for selected assets:

n'intervient que sur les objets sélectionnés

permet de creer des controller supplementaires pour les assets (hors objets riggés avec des bones).

peut etre utilisé sur un mesh ayant deja un master.

il est destiné aux objets qui ne seront pas manipulés par les personnages, mais simplement agencés differemment d'une scene à l'autre ou d'une copie à l'autre.

cas d'usage: si l'objet sélectionné est enfant d'un autre mdl, il créé un ctrl spécifique pour cet objet en plus de creer un master pour l'ensemble.

un tool permettra bientôt de convertir les hierarchies de controllers en hierarchies de bones et ainsi permettre de faire des versions riggées d'assets préexistants.

process:

1. parenter les mdl entre eux pour definir la hierarchie, ou les inclure dans une collection spécifique à leur nom pour en faire une "sous-collection".
2. selectionner les mdl necessitant d'avoir leur propre sous-controllers
3. une fois executé, le script recrée une hierarchie avec un master controller au sommet et les sous-controller enfants necessaires

si l'objet sélectionné n'as pas de parent, crée uniquement un master controller

si tous les objets sont inclus dans une seule collection dédiée (à leur nom), tous les objets de cette collection, meme si ils ne sont pas parentés seront enfant du master.

détail des actions du script:

- crée un controller pour chaque objet sélectionné (permettant de modifier sa position/rotation/echelle d'un plan à l'autre)
- crée un master controller portant un nom adapté à l'objets (remplace "mdl" par "ctr" et ajouter le suffixe "mstr")
- reset le transform du ou des objets (mdl)
- aligne le controller en position sur l'objet
- parente le ou les objets au controller
- parente tout les controller au master
- si les object sont dans une sous-collection, tout les autres objets de cette collection sont parentés au master

clone selected props:

n'intervient que sur les master controllers sélectionnés.

- clone l'asset avec tous ses enfants
- renomme l'ensemble
- range dans des collections appropriées
- sélectionne le master controller de la copie

replace data copies:

ne fonctionne qu'avec les assets background.

copie les data de l'asset(mesh) sélectionné et les colle sur toutes les instances de l'asset dans la scene (identifiées par le nom + numero)

export selected props:

n'intervient que sur les master controllers originaux(numerotés 01) sélectionnés.

- repositionne temporairement l'asset en zero
- export directement au format fbx dans le repertoire "assets/resources" de unity
- remet l'asset à sa position initiale

player:

ouvre un player unity pour visualiser les assets depuis blender.

rigging panel:

clean rig:

clean les rigs.

drivers by default

Crée un ensemble de bones permettant de gerer les deformations aux articulations.

ils sont représenté par une petite sphere et porte le prefixe [CTR_]

Les bones en question se trouvent sur les articulations suivantes:

- coudes
- poignets
- hanches
- genoux
- chevilles Chaque articulation est composée, de 2 (coude/genoux),3 (hanches) ou 4 bones(chevilles,poignets). Chaque bone est animable par l'animateur si il le souhaite, mais ils sont surtout enfant d'un autre bone dont le prefixe est [HLP_] qui lui est paramétrable. Ainsi vous pouvez regler des preset de deformation pour chaque articulation qui satisferont les usages classiques du personnage, laissant à l'animateur la possibilité de controler la deformation pour les cas rares ou cela ne lui conviendrait pas.

reglages des articulations.

Pour pouvoir determiner les valeurs necessaires de preset, vous devez au préalable avoir une animation de test vous permettant de juger du resultat dans les differentes situations et avec de petites et grandes amplitudes de rotation.

Les parametres de reglage se trouvent sur chaque bone[CTR_] sous forme de properties. Ils sont accessibles dans le panneau de parametre item de la vue 3D, dans l'onglet properties.

les parametres se divisent en deux series: les "_front" et les "_back".

Ils vous permettent de donner une deformation differente, suivant le sens dans lequel l'articulation plie.

Vous avez le controle sur 3 axe de deplacement et un axe de rotation.

- axe x: le controller glisse le long du membre.
- axe y: le controller s'eloigne ou s'enfonce dans le membre.
- axe z: le controller se decale par rapport au membre.

symmetry drivers L -> R [↗](#)

symetrise les valeurs de preset des articulations de la gauche vers la droite du personnage

symmetry drivers R -> L [↗](#)

symetrise les valeurs de preset des articulations de la droite vers la gauche du personnage

curves by default [↗](#)

Remplace les bendy bones en vue de pouvoir exporter vers unity.

il faut donc NE PAS utiliser les bendy bones.

crée des bras et jambes que l'on peut courber en utilisant les bones TWK

drivers on selected bone [↗](#)

Inactif actuellement

save drivers param [↗](#)

Sauvegarde dans le repertoire du personnage les valeurs de preset des articulations, au format JSON.

load drivers param [↗](#)

Lit dans le repertoire du personnage les valeurs de preset des articulations, au format JSON.

export character [↗](#)

Export le personnage au format fbx, sans animation (meme si il en a une)

ne fonctionne que si il y a une production definie de sorte que le chemin d'export soit defini (par default: sur UNITY\Assets\Resources\Common\ch)

layout panel: [↗](#)

update [↗](#)

un ensemble de boutons dédié à pouvoir updaten la scene ou les fichiers viendra bientôt.

populate [↗](#)

lorsque vous ouvrez une tache pour la premiere fois, generez le fichier blender à partir de la breakdown list en cliquant sur le bouton "POPULATE".

une fois récupéré tout les elements de la scene dans les fichiers .blend (personnages, props, bg) la scene est updaté avec les fichiers setdress et cbx si ils existent.

l'ensemble (fichier blender de la tache et setdress à jour) sont sauvegardés et les setdress commit sur svn.

populate all [↗](#)

cette fonction accessible uniquement aux membres de la production permet de preparer tous les shots en une fois (fichiers setdress) à partir de la breakdownlist.

open database: [↗](#)

(mise en place à venir ultérieurement)

Tout les soirs la database de la prod est mise à jour. vous y avez acces par le bouton open database.

Pour faire une recherche il faut indiquer l'information que vous chercher et sa categorie:

les categories:

- **type** correspond à des tags prevus. à ce jour: [BG, CH, PR]
- **name** correspond à tout ou partie du nom de l'asset
- **ext** correspond à l'extension. à ce jour: [fbx]
- **prod** correspond à la production. à ce jour: [tchouks, castor]
- **episode** correspond au numero de l'episode. si c'est hors episode, le numero est 000

pour l'instant seul les recherches par nom ou partie de nom sont possible.

vous pouvez utiliser les operateur AND et OR pour affiner votre recherche.

- **AND** precise votre requete. les reponse devront valider les 2 informations (exemple: name=vine AND name=hoop)
- **OR** elargie votre recherche. les reponses devront valider l'une ou l'autre des 2 informations

dans la liste des reponses qui apparaissent, cliquez sur le nom au dessus de l'iamge pour merger directement l'asset dans la scene.

create rigged camera: [↗](#)

crée une camera prete à l'animation et l'esportation vers unity.

remarque:

les cameras et lights sont considérés et donc nommées comme des props

export setdressing: [↗](#)

créé un fichier texte dans le repertoire "assets/Resources" contenant la liste des master controller presents dans la scene blender et leur transform (position , rotation, scale).

import setdressing: [↗](#)

import d'un setdressing.

mbx export material: [↗](#)

export d'un fichier *.mbx decrivant tout les parametres communs blender/unity de tous les materiaux de tout les assets presents dans la scene.

anim panel: [↗](#)

interface anim: [↗](#)

ouvre une interface d'animation.

update characters list: [↗](#)

met à jour la liste des personnages inclus dans la scene.

cbx export anim: [↗](#)

export l'anim du rig sélectionné.

cbx import anim: [↗](#)

import l'anim du rig sélectionné.

export facial anim: [↗](#)

export l'anim des bones speciaux d'un rig sélectionné.

versions panel: [↗](#)

change version: [↗](#)

supprime un asset pour le remplacer par une versio choisie du meme asset.

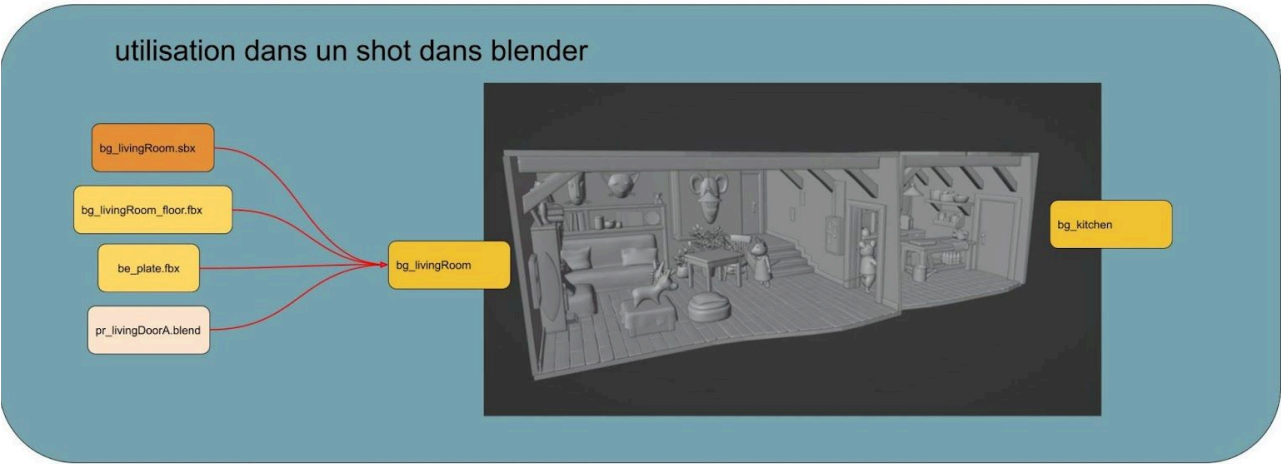
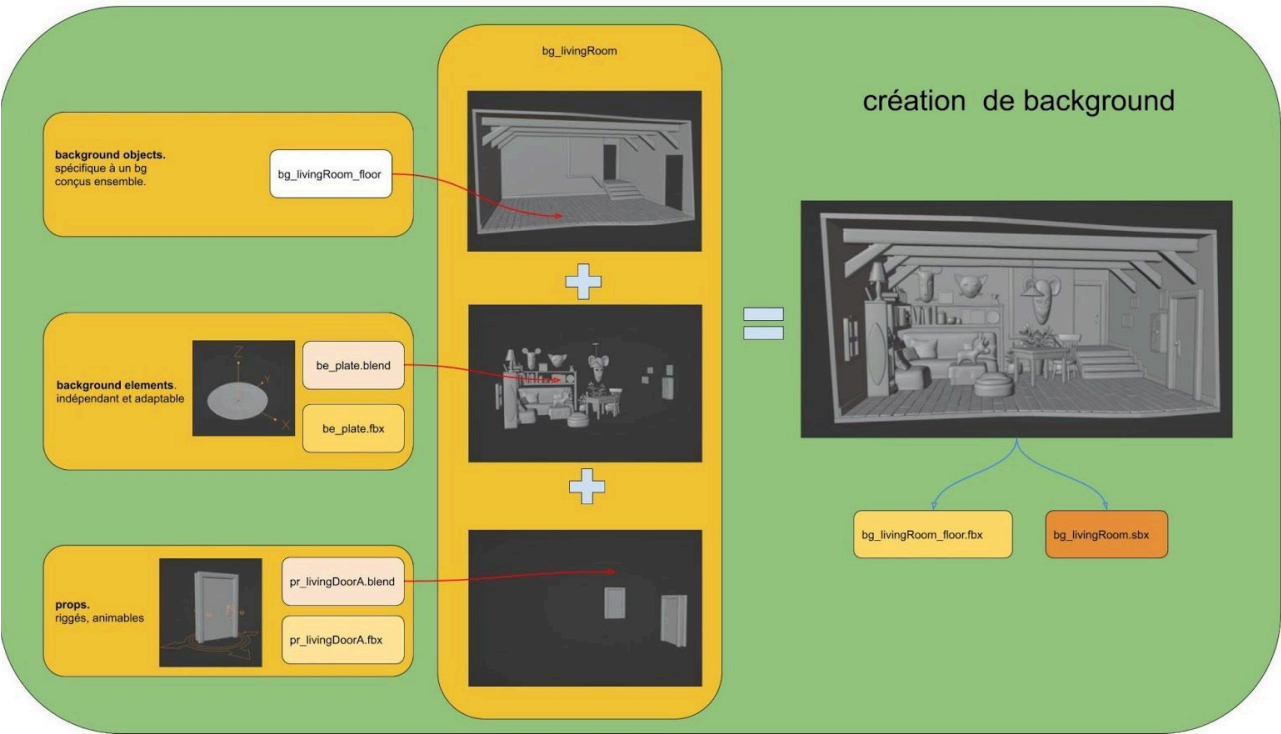
nomenclature rig: [↗](#)

les noms de layers sont placés au debut, séparé du nom par un tiret("-")

les infos de lateralités sont placées à la fin. ("_L" ou "_R")

annexes: [↗](#)

schéma de création des background: [↗](#)



background creation scheme: [↗](#)

vocabulaire: [↗](#)

asset: une ou plusieurs géométries et les controllers qui les réunissent. ils en existent plusieurs types:

- type bg: asset statique avec des controller "empty", dédié à un décor particulier.
- type be (bg-éléments): asset statique avec des controller "empty", multi usages .
- type pr (props): asset animable avec des rig de bones "direct" (les bones controllers sont aussi deformers), réutilisable. à noter que les caméras et lights sont de ce type.
- type ch (characters): personnage avec des rig "indirectes" (les bones controllers et deformers sont distincts).

d'autres types peuvent être utilisés: fx(fx), matte (mt).

shot: un plan d'un épisode.

séquence: un ensemble de plans utilisant le même décor (worldmap)

take: version validée d'un asset ou d'un shot.

bg: un décor dans espace défini. (ex: une pièce d'appartements, un jardin...) composé d'assets de type background (bg), background-elements (be), props (pr), tous enfants d'un unique "root" en son centre.

→ à noter: bg désigne donc à la fois un type d'assets et un assemblage d'assets de différents types.

worldmap: un plan décrivant l'agencement de plusieurs bg dans un espace donné. (ex: un palier et les différents appartements auxquels on peut accéder). il liste la position de chaque root de bg. Il existe des worldmap "génériques" décrivant l'agencement des décors par défaut, et des worldmap "séquences" décrivant la variation faite d'une générique pour une séquence donnée. Chaque bg est identifié par un empty dont le nom est affiché à l'écran, et sur lequel on merge un asset lowpoly donnant le volume et les formes principales du bg (mur,sol,etc..).

SVN: ne concerne que la fabrication d'assets.

un système de database et de versionning de fichiers partagés. Chacun récupère (update) la dernière sauvegarde du fichier et dépose (commit) sa propre sauvegarde après modification.elle devient la dernière sauvegarde en date. Les sauvegardes sont commentées par l'opérateur et les sauvegardes précédentes restent accessibles via la database. ainsi tout le monde semble travailler sur le même fichier.

fonctionnement des collections: ↻

les collection sont au minimum: une collection "bg" avec une sous collection "bg_(nom+num)", avec une sous collection "bg_(nom+num)mdl(extrainfo+num)*". les collection "mdl" peuvent contenir ou pas un 4eme champs. (voir exemples plus bas)

tout ce qui sera contenu dans une collection dont le nom est precedé du prefix "WIP_" ne sera pas pris en compte par les outils . cela permet d'exclure une partie de la scene des traitements automatiques tels que la creation des controller ou le check de nomenclature

au sein d'un bg, une sous collection permet de rassembler des ensembles d'objets.

si ces objets possèdent un 4ème champs commun (ex: bg_garden01_md1_tree01_trunc et bg_garden01_md1_tree01_leaves) un seul controller master sera créé pour l'ensemble des objets de la collection. si par contre au moins un objet de la collection à un 4ème champs distinct des autres, tous les objets auront leurs propres controllers master.

d'une manière générale, les collections sont une variantes des layers. elles servent à regrouper ensemble des objets non-parentés pour faciliter la lisibilité de la scène pour l'utilisateur. elles ne possèdent aucune information de transformation (et donc d'animation).

dans chaque collection ["pr/bg/ch"] se trouve au moins une collection regroupant tous les éléments, et nommée ["pr/bg/ch"][nom+num] (pour pouvoir regrouper tous les éléments de même type dans la même collection en cas de merge. exemple: si on merge plusieurs bg dans une même scène, ils seront tous dans la collection "bg", chacun avec sa propre collection ["bg"][nom+num])

dans chaque collection ["pr/bg/ch"][nom+num] se trouve plusieurs collections rassemblant tous les éléments par fonction (mdl/ctr/rig). comme tout asset possède au moins 1 controller, il y a au moins 2 collections nommées ["pr/bg/ch"][nom+num]["mdl/ctr"]

en cas de multiple sous parties, on peut rajouter le nom de la partie

dans chaque collection ["pr/bg/ch"][nom+num]["mdl/ctr"], si un ensemble d'éléments appartiennent au même asset, ils sont regroupés dans une collection nommées ["pr/bg/ch"][nom+num]["mdl/ctr/rig"]_["nomAsset + assetNumVersion(*)"]

les collections ["mdl"] et ["ctr"] sont donc construites en miroir et incluses dans une collection au nom de l'asset. Toutefois, un asset de bg peut contenir plusieurs modé regroupé dans une sous collection et ne possédant qu'un controller master pour tous (ex: un arbre) auquel cas on ne crée pas de collection pour le controller en miroir de la collection des mdl.

notice de creation des json des rigs

un fichier json doit se trouver dans un sous_repertoire "interface" du repertoire ou se trouve le personnage, il doit porter le nom du rig.

au meme endroit doit se trouver une image au format png du personnage (format carré)

l'image porte le nom du rig+ "_interface".

exemple:

le personnage s'appelle ch_chefProj01

le rig: ch_chefProj01_rig

le json: ch_chefProj01_rig.json

l'image: ch_chefProj01_rig_interface.png

divers paragraphes sont à intégrer dans le json pour que l'interface puisse être utilisée avec le personnage certains sont obligatoires, d'autres optionnels:

pour simplifier le travail, il est préférable de laisser la nomenclature rigify telle qu'elle est générée automatiquement. cela permettra de faire un maximum de copier/coller pour fabriquer les json à partir de ceux déjà faits.

- **"Name"**: le nom du rig
- **"Rig_Id"**: le numero identifiant du rig (custom propriété de l'armature)
- **"Layers"**: la liste des numeros de layers pour chaque partie du personnage exemple: "Left_Leg": {"IK":8,"FK":9,"TWK":10}, lorsque l'interface contient un switch (bouton radio) entre les différents layers, chaque

bouton est listé avec associé son numero de layer (ex: "ik":8) lorsque le layer est unique et n'est pas susceptible d'etre changé, le mot clé est "NULL" dans certains cas, le layer peut etre changé, mais sans etre accessible via un switch à son nom present dans l'interface (changement occasionné par un autre switch) , dans ce cas le mot clé est libre (tout sauf "NULL") exemple, un changement d'etat du personnage peut amener à changer un ou plusieurs layers pour acceder à d'autres versions du personnage et donc d'autres mesh et bones rangés dans des layer à part)

- **"AlternativeLayers"**: dans les cas cités plus haut, des layers peuvent en remplacer d'autres. ils sont listés ici avec leur nom et le numero correspondant. exemple: "Facial":
{ "FORWARD":3, "BACKWARD":4, "rightFORWARD":19, "rightBACKWARD":20 } on retrouve les noms et numeros present dans "Layers" plus ceux pouvant se substituer.
- **"Parts"**: pour chaque partie listée dans "Layers", la liste des bitmap servant de bouton pour construire le pantin. les bitmap sont dans les tools (ex:08_TOOLS\ADDONS\BlenderTools\qtutils\bitmap) exemple: "Left_Leg":
["S_thigh_L", "knee_L", "calf_L", "ankle_L", "foot_L", "footIK_L", "toe_L"],
- **"Bones"**: pour chaque partie, un ou plusieurs layers sont possibles contenant chacun un ensemble de bones. ils sont listé ici, sous un mot clé composé du nom de la partie + le nom du layer. exemple: "Left_Leg_IK":
["thigh_ik.L", "thigh_ik_target.L", "NULL", "foot_spin_ik.L", "foot_heel_ik.L", "foot_ik.L", "toe.L"], "Left_Leg_FK":
["thigh_fk.L", "NULL", "shin_fk.L", "foot_fk.L", "NULL", "NULL", "toe.L"], le nombre de bones listé doit etre egal au nombre de bitmap correspondant dans la partie "Parts". si une bitmap dans "Parts" ne correspond à aucun bone selectionnable dans le layer en question, il est remplacé par "NULL".
- **"Toggles"**: dans les cas particulier des ik/fk, le bouton de switch entre les deux etats est accessible dans rigify depuis un bone particulier. le nom de ce bone est listé ici. exemple: "Left_Leg": "thigh_parent.L",
- **"Switchs"**: la liste des switch (groupe de boutons radio) present dans l'interface.
- **"ChangingParts"**: certains boutons peuvent donner accès à une liste de modification. On peut par exemple remplacer un mesh par un autre (switch de visibilité) ou une texture par une autre. c'est ici que sont listé les ressources nécessaires. Les changements possibles apparaitront dans la colonne de droite de l'interface.
- **"SpecialToggles"**: certains boutons radio peuvent enclencher des actions complexes. ils sont listés ici.
- **"aligns"**: la liste des bones nécessaires aux fonction d'alignement ik > fk et fk > ik en cas de problème, elle peut être récupéré dans le script rig_ui.py associé au rig (section scripting de blender)

descriptif des scripts composants le pipe:

tout se trouve dans C:\Users*username*\AppData\Roaming\Blender Foundation\Blender\3.1\scripts\addons\ManetteTools et dans le sous répertoire qtutils.

les scripts:

- **manetteUI** = interface generale utilisateur dans blender
- **daba** = interface d'accès à la base de donnée (appelé par manetteUI)
- **check_database** = script de check auto et mise à jour de la base de donnée (lancé par autoDB.bat)
- **interface_aquarium** = routines pour les acces à aquarium / shotgun / fichiers de config (appelé par manetteUI)
- **clean_scene** = script pyside2 pour le panneau de clean de scene
- **clean_func** = ensemble de fonctions pour nettoyer les scenes
- **sanity_check** = script pyside2 pour le panneau de sanity_check
- **sanity_func** = ensemble de fonctions appelées par le sanity_check
- **interface_anim** = script de l'interface d'animation
- **rig_tools** = fonctions necessaires au rigging et à l'export des rig
- **recursiveLinks** = routines recursives.
- **nomenParsing** = routines de traitement de la nomenclature
- **qtutils\core** = class de fenetre pyside2 qui sera instancié dans blender pour générer l'interface

- **dabaqtutils\prodDef.json** = les variables définissant les chemins de toutes les prods
- **qtutils\prodParsing.json** = les chemins que la base doit parcourir pour se mettre à jour
- **qtutils\prodDB.sqlite** = la base de donnée en elle meme
- **qtutils\pyside2.bat** = installateur de pyside2

- **fonctionnement de la base de données:**

la base est directement accessible dans blender depuis le ManetteUI, et elle permet de merger directement l'asset selectionné dans la scene.

le fonctionnement des mises à jour auto de la base: [🔗](#)

le fichier autoDB.bat lance automatiquement un blender et le script check_database.py. celui ci parcourt les repertoires indiqués dans prodParsing.json et verifie l'etat de la base et la met à jour si necessaire. la base est faite de deux tables, la premiere, "assets" contient 3 colonnes: id, path et Thumb correspondant au chemins de chaque asset et du thumbnail correspondant. la deuxieme table "tags" contient 4 colonnes: id , assetID, tagType, tagName.les infos stockées pour l'instant concerne uniquement des fbx, et sont Type, name, ext, prod et episode.le script verifie l'existence du thumbnail correspondant au fbx qu'il trouve, si il ne le trouve pas ou qu'il est plus ancien que le fbx, il en refait un et remet à jour la base.dans un deuxieme temps il parcourt la base pour verifier qu'il n' y pas de fbx listé qui n'existerait plus, auquel cas il fait le menage dans la base. au terme du process il commit sur svn la base et les thumbnails nouvellement créés.

- **tips:**

pour éviter de voir des objets pris en compte par les outils du pipe, on peut les ranger dans une collection portant le préfixe("wip_")